Reproducing Multi-Agent System Failures: ChatDev 1.0 and Magentic-One under gpt-5.4-mini

MAS memory research — working report

June 10, 2026

Abstract

We reproduce 30 tasks from the MAST study ("Why Do Multi-Agent LLM Systems Fail?") on two multi-agent systems — ChatDev 1.0 (ProgramDev) and Magentic-One (GAIA) — running their *native, unmodified* codebases with the original model (GPT-4o) swapped for gpt-5.4-mini behind an API-translation proxy. All 60 traces of interest (30 reproduced; originals for calibration) are labeled by a two-tier LLM judge (gpt-5.5): a taxonomy-blind close reading producing open-ended findings, followed by evidence-quote-required MAST labels. Headline results: the selected failure set largely survives the model upgrade (Magentic-One 2/15 $\rightarrow$ 3/15 solved; ChatDev 3/15 $\rightarrow$ 5/15); ChatDev exhibits Information Withholding (2.4) in *every* run including successful ones, rooted in a lossy code-persistence channel; Magentic-One shows the opposite signature (2.5-heavy, 2.4-light), consistent with its star topology. The open-ended judge tier surfaces mechanism chains the taxonomy cannot express.

1 Introduction

MAST [1] introduced a taxonomy of 14 multi-agent failure modes in three categories (specification, inter-agent misalignment, verification) and annotated traces from several MAS. Our research interest is category 2 (inter-agent misalignment), specifically whether its modes are *capability* failures that fade with stronger models or *structural* failures rooted in the systems' communication and memory architecture. This report documents the first experimental step: faithfully reproducing a biased sample of MAST tasks on a current small model and re-labeling old and new traces with a stronger judge.

2 Task dataset construction

The dataset is deliberately biased and is not a representative benchmark sample. Tasks were selected to maximize the prior probability of observing inter-agent misalignment, with controls retained for contrast. Selection proceeded in three steps:

1. **Provenance recovery.** The public per-trace MAST annotations (HF `mcemri/MAD`) are unusable: only 206 unique annotation rows exist for 1,242 traces and the annotation is purely a function of the row index (verified across both dataset revisions). We therefore recovered ChatDev human annotations from a deleted file in the MAST repository's git history (32 game tasks; sparse labels; three tasks carry explicit category-2 marks: TicTacToe 2.1, Wordle 2.2, Sudoku 2.2), and computed Magentic-One outcomes locally by comparing each trace's final answer against the GAIA gold answer (120/165 original runs failed).

2. **Screening for category-2 priors.** Every candidate failed task's *original GPT-4o trace* was close-read and classified for category-2 symptoms with quoted evidence. Tasks whose original failure had no inter-agent surface (e.g. solved single-shot by the orchestrator) were excluded.

3. **Composition.** 15 ChatDev tasks: 4 high-likelihood cat-2, 4 medium, 4 low (non-cat-2 failure contrast), 3 originally-solved controls. 15 Magentic-One GAIA tasks: 1 high, 9 medium, 3 low, 2 solved controls (8×L1, 6×L2, 1×L3; image-attachment tasks excluded).

3 The two architectures and the reproduction setup

3.1 ChatDev 1.0

A waterfall "software company": fixed phase chain (DemandAnalysis → LanguageChoose → Coding → CodeCompleteAll → CodeReview×3 → Test → EnvironmentDoc → Manual), each phase a two-agent dialogue between role-prompted instances (CEO, CTO, Programmer, Code Reviewer, Test Engineer). Inter-phase memory is a *shared code store*: a regex extractor parses fenced code blocks out of chat messages and persists them; phases pass forward short summaries plus the store. Two properties matter for what follows: (i) completion budgets are computed as 4096 − prompt tokens for gpt-4o, so long emissions are truncated mid-file, and the system never checks the truncation signal; (ii) the extractor silently drops malformed or unterminated code blocks.

3.2 Magentic-One

A star topology: an orchestrator maintains a task ledger (facts/plan) and a progress ledger, delegating single steps to four specialists (WebSurfer — multimodal browsing over screenshots; FileSurfer; Coder; ComputerTerminal) and re-planning when progress stalls. All inter-agent information flows through the orchestrator; re-plans rewrite the ledger from the orchestrator's summary of history.

3.3 Reproduction setup

Both systems run their official code: ChatDev at tag v1.1.6 (last 1.x release, within the MAST run window), Magentic-One via the MAST authors' own agbench harness — each original trace directory ships the exact `scenario.py` executed, byte-identical to the agbench template at `microsoft/autogen@af5dcc7` we re-execute it verbatim with AutoGen pinned to 0.4.8. The only swap is the model endpoint: a local proxy translates the OpenAI chat-completions protocol both systems speak to Perplexity's Responses API serving `gpt-5.4-mini`, aliasing the configured model name "gpt-4o" (zero patches to either codebase; tool calls, JSON mode, and — verified — screenshot vision all pass through). Known deviations, all documented in the repo: host-local code execution rather than Docker; `max_tokens` dropped when ChatDev computes it ≤ 0 (original behavior is a fatal API error, not a different result); GAIA web drift since 2024 (answers are time-anchored).

4 Outcomes: did the failures reproduce?

The original MAST runs used GPT-4o (their judge was o1; outcome comparisons here are against the GPT-4o *runs*). Table 1 lists per-task outcomes.

Aggregates: Magentic-One 2/15 → 3/15 solved (one original failure now passes; no regressions; both controls reproduce). ChatDev 3/15 → 5/15 (CandyCrush and TextBasedSpaceInvaders now judged working; all three controls reproduce). Two caveats: ChatDev graders differ across eras (human

Task	cat-2 prior	GPT-4o (orig.)	gpt-5.4-mini (repro.)
ChatDev (success per human annotation / per our judge)			
CandyCrush	low	fail	pass
Checkers	low	fail	fail
ConnectFour	control	pass	pass
Connections	medium	fail	fail
DouDizhuPoker	high	fail	fail
Gomoku	control	pass	pass
MonopolyGo	low	fail	fail
Pong	control	pass	pass
Strands	medium	fail	fail
Sudoku	high	fail	fail
TextBasedSpaceInvaders	high	fail	pass
The Crossword	low	fail	fail
TicTacToe (with display)	medium	fail	fail
Tiny Rouge	high	fail	fail
Wordle	medium	fail	fail
Magentic-One (success by normalized exact match on GAIA answer)			
00d579ea (L3)	medium	fail	fail
023e9d44 (L2)	medium	fail	fail
0383a3ee (L1)	control	pass	pass
04a04a9b (L2)	low	fail	fail
05407167 (L2)	medium	fail	fail
08cae58d (L2)	medium	fail	fail
27d5d136 (L1)	control	pass	pass
2b3ef98c (L2)	low	fail	fail
366e2f2b (L2)	medium	fail	fail
3cef3a44 (L1)	medium	fail	fail
3f57289b (L1)	low	fail	fail
5a0c1adf (L1)	high	fail	fail
5d0080cb (L1)	medium	fail	pass
72e110e7 (L1)	medium	fail	fail
7673d772 (L1)	medium	fail	fail

Table 1: Per-task outcomes, original MAST runs (GPT-4o) vs. our reproduction (gpt-5.4-mini).

annotation then vs. our judge now), and GAIA answers are time-anchored, so some Magentic failures may be web-decay rather than coordination; the per-case briefs in Appendix B distinguish these. The substantive conclusion: **the failure set largely survives a two-generation model upgrade**, consistent with structural rather than purely capability-bound failure causes.

5 MAST taxonomy statistics

5.1 Judge

Labels come from a two-tier gpt-5.5 judge (temperature 0). Tier 1 reads the trace with no taxonomy and reports open-ended findings (description, agents, verbatim quote, impact, possibly-innocent flag, free-form kind). Tier 2 receives the MAST definitions and canonical examples plus tier-1 findings and emits one binary decision per mode, each requiring a verbatim evidence quote (flags without quotes are dropped programmatically). Calibration on four ground-truth anchors (the

three human-labeled ChatDev tasks plus one screening-verified 2.4/2.5 case): commission modes (2.3–2.6, 3.x) are detected reliably — notably 2.4/2.5, which both the MAST authors' o1 judge and our earlier gpt-5.4 judge emitted *zero* times across all traces — while omission modes reach only ≈50% recall (2.1 and 2.2 figures below are lower bounds). Precision against sparse human labels is not measurable; every flag carries its quote for case-level verification.

Mode	Name	ChatDev (n=15)	Magentic-One (n=15)
1.1	Disobey Task Specification	15	14
1.2	Disobey Role Specification	0	2
1.3	Step Repetition	14	10
1.4	Loss of Conversation History	10	2
1.5	Unaware of Termination Conditions	4	4
2.1	Conversation Reset	7	4
2.2	Fail to Ask for Clarification	5	8
2.3	Task Derailment	6	8
2.4	Information Withholding	15	4
2.5	Ignored Other Agent's Input	15	13
2.6	Action-Reasoning Mismatch	12	13
3.1	Premature Termination	13	13
3.2	No or Incorrect Verification	13	13
3.3	Weak Verification	13	13

Table 2: MAST failure-mode incidence in the 30 reproduced traces (gpt-5.5 judge, evidence-quote required per flag). 2.1/2.2 are lower bounds (Section 5).

5.2 Architecture signatures

The two systems show opposite category-2 profiles (Table 2). ChatDev flags 2.4 *and* 2.5 in 15/15 traces — including the runs that produced working software — because the code-persistence channel loses artifacts in every run; whether the task survives depends on *which* artifacts are lost. Magentic-One is 2.5-heavy (13/15) but 2.4-light (4/15): in a star topology the hub sees every message, so little can be withheld, but the hub routinely discards or overrides specialist input. Verification modes (3.1–3.3) are near-universal in both systems. These signatures match the architectures, supporting the view that category-2 incidence is a property of the communication topology rather than of the underlying model.

6 Open-ended judge trends

Tier-1 produced ∼650 findings across the 30 traces, each with a judge-coined free-form label. Table 3 shows the recurring clusters (reproduction-harness artifacts — model-name warnings, visualizer logging noise, timestamp formats — are excluded).

Three mechanism chains dominate and are invisible to the binary taxonomy:

1. **ChatDev: the lossy persistence chain.** Budget-truncated code emission → extractor silently drops the unterminated file → shared store goes stale → reviewer repeats an identical comment for three cycles → tester rubber-stamps → manual documents features that do not exist. Token accounting confirms the root cause: completions hit ChatDev's 4096 − prompt budget exactly. The clusters *truncated output/code/patch*, *lost artifact*, and *state corruption* are this chain observed at different stages.

Judge-coined finding cluster	ChatDev traces	Magentic traces
scope drift	11	0
state corruption	10	1
redundant reflection	8	0
unsupported final	0	7
truncated manual	6	0
truncated output	6	0
lost artifact	5	0
truncated code	5	0
empty context	5	0
unused tools	0	5
premature conclusion	4	0
weak testing	4	0
shallow testing	4	0
missed verification	0	4
format artifact	3	0
truncated patch	3	0
code corruption	3	0
truncated fix	3	0
format violation	3	0
lost file	3	0
repeated review	3	0
format drift	3	0
format looseness	3	0
configuration warning	0	3
unused tool	0	3
blocked search	0	3
partial extraction	0	3

Table 3: Recurring open-ended (taxonomy-free) finding clusters across the 30 reproduced traces; counts are distinct traces. Reproduction-harness artifacts excluded.

2. **ChatDev: error masking.** When a missing artifact finally breaks the build, the programmer patches *around* it (try/except fallbacks, `pip install` of an unrelated package matching the missing module's name) instead of re-emitting the artifact it produced earlier — converting a visible failure into an invisible one.

3. **Magentic-One: unsupported finals and unused tools.** The orchestrator answers from prior knowledge or partial evidence (*unsupported final*, 7 traces) and skips planned delegation (*unused tools*, 5 traces); ledger re-plans drop working state. The *ignored input* mode is usually the hub overriding a specialist's correct partial result.

A Case briefs: ChatDev

ChatDev: CandyCrush

Task. Implement a match-3 puzzle game reminiscent of Candy Crush. Represent the board and let users swap adjacent candies to form matches of three or more. Matches are cleared, new candies appear, and scoring is tracked. The board updates after each valid move. Incorporate chain reactions when candies fall and possibly track moves or time-based constraints.

Outcome. Original GPT-4o run: failed (human annotation). Reproduction: solved (judge). Screened cat-2 prior: low.

MAST modes (judge). 1.1, 2.4, 2.5, 2.6, 3.1, 3.2, 3.3

Run narrative (judge, tier 1). The run selected an Application modality and Python, generated a Tkinter match-3 game with board logic, then extracted only two code files while silently reshaping the programmer's intended file structure: the original small main.py plus game.py architecture became a single main.py containing the GUI and a board.py. The initial implementation was plausibly runnable and covered swaps, matching, gravity/refill, scoring, moves, and timer, but review cycles became confused by distorted/stale code representations: the reviewer identified a real model-robustness issue, the programmer's attempted fixes were truncated and syntactically incomplete, later reviewers reviewed stale or corrupted code and sometimes missed severe defects, and no clear code update occurred after those attempted modifications. The test phase nevertheless reported success, apparently against the earlier extracted code, while requirements and manual generation used stale code/context; the generated requirements content was not propagated into the manual phase, and the manual itself was truncated mid-command. Overall, the core game appears to have been produced, but information flow between agents and phases was noisy, lossy, and inconsistent, with several potentially serious artifacts masked by a passing test report.

Failure summary (judge, tier 2). The system produced a mostly functional match-3 game, but code/artifact state was corrupted and truncated across review cycles, reviewer fixes were not applied, generated requirements were lost before the manual, and verification was shallow/incorrect.

Mechanisms flagged.

- *file loss*: The programmer initially designed a three-code-file structure with main.py importing game.py, but only two code files were ultimately counted/updated.
- *misextraction*: The code extraction step wrote the GUI code into main.py while its docstring still identified it as game.py.
- *silent rewrite*: The updated main.py diff shows game.py content, not the originally proposed main.py entry point.
- *distorted handoff*: The code reviewer was given the already-distorted code representation, with main.py labeled but containing the game.py docstring.
- *truncated patch*: The programmer's first review-modification response was truncated and syntactically incomplete.
- *bad state carryover*: Despite the malformed modification, subsequent state included a truncated modification_conclusion and corrupted-looking code fragments.

ChatDev: Checkers

Task. Develop a Checkers (Draughts) game. Use an 8x8 board, alternate turns between two players, and apply standard capture and kinging rules. Prompt for moves in notation (e.g., from-to positions) and update the board state accordingly.

Outcome. Original GPT-4o run: failed (human annotation). Reproduction: failed (judge). Screened cat-2 prior: low.

MAST modes (judge). 1.1, 1.3, 1.4, 1.5, 2.1, 2.3, 2.4, 2.5, 2.6, 3.1, 3.3

Run narrative (judge, tier 1). The run selected an Application modality and Python, then asked the Programmer to build a GUI checkers game. The Programmer initially described a two-file Tkinter implementation, but the ‘checkers_game.py‘ code was cut off mid-statement and the code extraction/update step only saved ‘main.py‘, leaving an import of a nonexistent local module. The Code Reviewer correctly identified the missing ‘checkers_game.py‘, but subsequent modification outputs were also truncated or not merged into the project state, and later review/test cycles repeatedly lost or ignored attempted fixes. Testing consistently failed with ‘ModuleNotFoundError: No module named ’checkers_game’‘; the system nevertheless tried to resolve this by ‘pip install checkers_game‘, produced contradictory error summaries saying nothing needed to be done, proceeded to requirements and manual generation, and ended with only one code file and a manual that references a required ‘checkers_game.py‘ that was never delivered.

Failure summary (judge, tier 2). The system repeatedly lost or failed to incorporate the required ‘checkers_game.py‘ module, ignored reviewer/test evidence about the missing module, repeated ineffective fixes, and finalized documentation for an application that still failed with ‘ModuleNotFoundError‘.

Mechanisms flagged.

- *truncated code*: The Programmer initially claimed a complete two-file implementation, but the generated ‘checkers_game.py‘ was visibly cut off mid-line.
- *file extraction loss*: Despite the Programmer outputting both ‘main.py‘ and ‘checkers_game.py‘, the code update step only created/updated ‘main.py‘.
- *truncated fix*: The first code-review modification attempted to add ‘checkers_game.py‘, but that output was also truncated mid-function.
- *format noncompliance*: The modification phase did not return all full project files as requested; it output only ‘checkers_game.py‘ even though the prompt requested complete codes.
- *merge failure*: After the attempted modification, project state still reported only one code file and 160 lines, indicating the new ‘checkers_game.py‘ was not incorporated.
- *state corruption*: State became corrupted or distorted in later placeholders: the stored ‘main.py‘ shown to review had serious mutations such as ‘def init‘, empty event bindings, incorrect ‘pos_to_notation‘ calls, and ‘if name == ”main”‘.

ChatDev: ConnectFour

Task. Implement a standard Connect Four game. Two players alternate placing discs into a seven-column, six-row grid. The objective is to form a horizontal, vertical, or diagonal line of four discs. Players choose columns by typing a number, and the board updates after each move until a win or draw. Validate column choices and end the game when a winning line is detected or the grid is full.

Outcome. Original GPT-4o run: solved (human annotation). Reproduction: solved (judge). Screened cat-2 prior: control.

MAST modes (judge). 1.1, 1.3, 1.4, 2.2, 2.3, 2.4, 2.5, 2.6, 3.2, 3.3

Run narrative (judge, tier 1). The run selected an Application modality and Python quickly, then the Programmer initially produced a reasonable multi-file Tkinter GUI Connect Four im-

plementation. However, the code-extraction/update step silently corrupted the generated files by saving ‘connect_four_ui.py‘ as only ‘python main.py‘, dropping the full UI code. The Code Reviewer correctly identified that fatal missing-UI problem, but its suggested fix shifted the project from the required/injected GUI direction to a console UI, and the Programmer implemented that console version. Subsequent review cycles were noisy and partly contradictory: one review called the win logic incorrect while also explaining it was correct, another asserted likely win-detection problems without evidence and asked for tests. The Programmer attempted to add a unit-test file, but the test code was truncated and never entered the saved code set; nevertheless the system reported tests passed. The requirements phase produced a standard-library-only ‘requirements.txt‘, but that information was not passed into the manual phase, whose requirements placeholder was empty. The final saved code appears to be a playable console Connect Four game satisfying the user’s typed-input task, but it no longer has the GUI that the system configuration/coding prompt required, and several artifacts/tests/dependencies were lost or inconsistently represented.

Failure summary (judge, tier 2). The system ultimately produced a playable console Connect Four game, but it lost the initial GUI implementation, drifted from the injected GUI requirement, repeated/restarted documentation work, gave contradictory and superficial reviews, and performed weak/incorrect verification.

Mechanisms flagged.

- *extra requirement*: The coding prompt injected a GUI requirement because ‘gui_design‘ was true, even though the original user asked for typed column numbers and did not explicitly request a GUI.
- *format violation*: The Programmer output extra prose, run instructions, and optional feature offers around the code despite the strict file-code-block format request.
- *code extraction loss*: The code update step silently corrupted ‘connect_four_ui.py‘, replacing the full GUI source with the shell command ‘python main.py‘.
- *lost information*: The full Tkinter UI information generated by the Programmer was lost before review; the reviewer only saw the corrupted file content.
- *requirement drift*: The Code Reviewer suggested implementing a console UI and even inferred that the codebase suggested console rather than GUI, ignoring the earlier explicit GUI requirement and the lost Tkinter code.
- *GUI dropped*: The Programmer followed the reviewer’s console suggestion, replacing the GUI design with a console interface.

ChatDev: Connections

Task. Develop a puzzle game where the player must group 16 words into four sets of four, based on hidden categories. The words are presented in a 4x4 grid, and players select four at a time to form a group. Correct groups are removed and revealed with a category and a color-coded difficulty (yellow, green, blue, purple). Incorrect guesses count as mistakes, with a maximum of four allowed. Only one correct solution exists, though words may appear to fit multiple categories. Include shuffle functionality and provide immediate feedback after each guess. A new puzzle is generated daily.

Outcome. Original GPT-4o run: failed (human annotation). Reproduction: failed (judge). Screened cat-2 prior: medium.

MAST modes (judge). 1.1, 1.3, 1.4, 2.4, 2.5, 2.6, 3.1, 3.3

Run narrative (judge, tier 1). The run selected an Application modality and Python, then the Programmer attempted a multi-file Tkinter implementation of a Connections-style daily word puzzle. The initial code response described six files, including puzzle and utility modules, but the actual extracted code only saved four files and omitted required modules, leaving unresolved imports; the reviewer correctly identified this, but subsequent modification was truncated and did not repair the project. Later review/test state became inconsistent and partly corrupted, the test found an ImportError from the missing/local-shadowed utils module, and the Programmer's attempted test fix produced only an incomplete utils.py stub that was not integrated. The system nevertheless proceeded to requirements and manual generation using stale/incomplete code; the requirements were not clearly saved as a file and the manual was itself truncated, so the final artifact remained non-operable.

Failure summary (judge, tier 2). The system produced and ended with an incomplete, non-runnable puzzle game after missing modules and truncated repairs were diagnosed but not integrated or reverified.

Mechanisms flagged.

- *missing files*: The Programmer claimed a multi-file implementation including ‘puzzles.py‘ and ‘utils.py‘, but only four files were actually updated, omitting both required modules.
- *circular import*: The initial generated ‘puzzles.py‘ design would have introduced a circular import: ‘game.py‘ imports from ‘puzzles‘, while ‘puzzles.py‘ imports ‘Puzzle‘ from ‘game‘.
- *truncated code*: The initial ‘puzzles.py‘ output was visibly truncated mid-list/string before completing the file.
- *silent omission*: The code extraction/update step silently accepted the incomplete generated response and updated only ‘main.py‘, ‘app.py‘, ‘game.py‘, and ‘models.py‘ without flagging missing referenced files.
- *unresolved imports*: The saved code referenced unresolved imports from missing modules.
- *dead feedback*: The game includes an ‘almost‘ feedback path in the GUI, but game logic always returns ‘almost=False‘ and never computes near-miss state.

ChatDev: DouDizhuPoker

Task. Implement Dou Dizhu (Chinese Poker) game for three players, one of whom is the 'landlord.' Cards must be played in valid combinations (singles, pairs, straights, etc.). The objective is to be the first to run out of cards or prevent the landlord from doing so, following standard Dou Dizhu rules. Include the bidding phase to determine the landlord, and enforce the pass-or-beat logic for played combinations.

Outcome. Original GPT-4o run: failed (human annotation). Reproduction: failed (judge). Screened cat-2 prior: high.

MAST modes (judge). 1.1, 1.3, 2.3, 2.4, 2.5, 2.6, 3.1, 3.2, 3.3

Run narrative (judge, tier 1). The run selected an Application modality and Python, then the Programmer attempted to generate a Tkinter Dou Dizhu implementation, but the generated GUI file was incomplete/truncated and never became a saved code file; only main.py and game_logic.py

were extracted. The Code Reviewer repeatedly identified the missing gui.py and inadequate bidding/UI flow, but subsequent Programmer modification attempts were truncated or failed to add the GUI, while state passed between phases was sometimes corrupted. Testing then found the expected ModuleNotFoundError, an inappropriate package-install-style fix was suggested, and the final code merely caught the missing gui.py and fell back to a minimal CLI demo, causing the shallow test to pass while the requested playable GUI/application, bidding phase, and full user interaction remained absent. Later documentation and requirements phases described or implied features that the final code did not actually provide, and the run ended with only two code files and a non-playable fallback.

Failure summary (judge, tier 2). The system repeatedly failed to create the required GUI/playable Dou Dizhu application, ignored review feedback to add gui.py, masked the missing module with a CLI fallback, and accepted a shallow launch-only test as success.

Mechanisms flagged.

- *missing GUI*: The Coding phase explicitly required a GUI, but the implementation that ultimately got saved did not include a GUI file.
- *truncated output*: The Programmer claimed to provide a complete functional Tkinter implementation, but the gui.py output was visibly incomplete/truncated.
- *lost file*: The system's code extraction/update step silently dropped gui.py and saved only main.py and game_logic.py.
- *rule simplification*: The implemented bidding logic was far simpler than standard Dou Dizhu bidding and immediately assigned the first bidder as landlord.
- *incomplete rules*: The combination logic was only a subset of standard Dou Dizhu despite the user asking for standard rules and 'singles, pairs, straights, etc.'
- *unfixed review*: The Code Reviewer correctly identified the missing gui.py and non-operable application, but this finding was not effectively acted on.

ChatDev: Gomoku

Task. Develop a standard Gomoku game. Implement it on a typical 15x15 board, where two players alternate placing black or white stones. A player wins by forming an unbroken row of five stones horizontally, vertically, or diagonally.

Outcome. Original GPT-4o run: solved (human annotation). Reproduction: solved (judge). Screened cat-2 prior: control.

MAST modes (judge). 1.1, 1.3, 1.5, 2.3, 2.4, 2.5, 2.6, 3.2, 3.3

Run narrative (judge, tier 1). The run selected an Application modality and Python, then the Programmer generated a reasonable three-file Tkinter Gomoku implementation, but the code-ingestion step immediately mis-extracted 'board.py' as the shell command 'python main.py', losing the actual Board implementation. Code review caught that fatal issue, but subsequent review/modification cycles became noisy: one Programmer modification response was truncated mid-'game.py', internal placeholders showed distorted code variants, a reviewer raised a largely spurious 'run()'/'mainloop()' operability issue while also signaling finished, and another reviewer later invented or overstated a coordinate-mapping issue. The system ultimately appears to retain

a runnable three-file implementation after the second review cycle and reports a test pass, but the last modification response was again truncated and apparently not applied, the test report is underspecified, the requirements phase produced only comments and then its result was dropped from the manual phase, and the final documentation was generated by re-deriving dependencies from code rather than consuming a created ‘requirements.txt‘ file.

Failure summary (judge, tier 2). The system ultimately produced a runnable Gomoku implementation, but suffered code extraction/information loss, unnecessary review/modification cycles, spurious review reasoning, and shallow or incorrect verification.

Mechanisms flagged.

- *format violation*: The coding response did not strictly follow the requested file-only markdown format; it included explanatory prose, a “How to run” section, and optional feature suggestions outside file code blocks.
- *code extraction loss*: The Programmer initially supplied a correct ‘board.py‘, but the code-update/extraction step replaced it with the unrelated command ‘python main.py‘.
- *lost information*: Information from the correct generated ‘board.py‘ was lost before review; the reviewer only saw the corrupted ‘board.py‘ and had to re-derive or request the Board implementation.
- *truncated output*: The first modification response was truncated in ‘game.py‘ immediately after the ‘_draw_stone‘ docstring, so the response did not contain complete code for all files as required.
- *partial update*: After the truncated first modification response, the system updated ‘main.py‘ and ‘board.py‘ but did not show an update for ‘game.py‘, silently retaining the older ‘game.py‘ with the ‘run()‘ method.
- *state distortion*: Internal placeholder state showed severely distorted code variants, including stripped double underscores and altered event binding, even though the visible reviewer prompt later showed less-corrupted code.

ChatDev: MonopolyGo

Task. Create a simplified version of Monopoly Go! with basic board mechanics: rolling dice, moving around properties, buying them, collecting rent, and handling chance events. Track player money and property ownership. Display relevant game information and prompt each player for decisions. Implement essential rules for jail, free parking, and simplified chance cards.

Outcome. Original GPT-4o run: failed (human annotation). Reproduction: failed (judge). Screened cat-2 prior: low.

MAST modes (judge). 1.1, 1.3, 1.4, 2.1, 2.3, 2.4, 2.5, 2.6, 3.1, 3.2, 3.3

Run narrative (judge, tier 1). The run chose an Application modality and Python, then attempted to generate a Tkinter Monopoly Go-style game. The initial coding response claimed to provide main.py, game.py, and model.py, but model.py was truncated mid-function and the system only persisted main.py and game.py, leaving imports to a nonexistent model module. Code review correctly identified the missing model.py and requested a complete implementation, but subsequent programmer revisions were themselves truncated or not integrated, and internal code state became

inconsistent/corrupted. Testing repeatedly reported ModuleNotFoundError for model, yet the error summary misleadingly said nothing needed doing, and the system even tried to fix a local missing source file by running pip install model. Later phases proceeded to requirements and manual generation using only the broken two-file codebase; the manual noted model.py was required, but the final software still listed only two code files and never passed tests.

Failure summary (judge, tier 2). The system produced and documented an incomplete Monopoly Go application whose required model.py was repeatedly generated or requested but never persisted, causing unresolved ModuleNotFoundError despite review and test phases.

Mechanisms flagged.

- *incomplete code*: The Coding prompt required complete functional code and no placeholders, but the Programmer's model.py output was visibly incomplete, ending mid-method.
- *claim mismatch*: The Programmer claimed to provide a complete runnable implementation, but the actual displayed model.py was truncated.
- *lost file*: The code extraction/update step silently dropped model.py entirely even though the Programmer response included a model.py section.
- *state inconsistency*: The system reported only two code files after initial coding despite game.py importing model.py.
- *failed revision*: In CodeReviewModification, the Programmer was told to return full complete codes, but returned only model.py, and that file was truncated at a syntax error.
- *update failure*: The attempted model.py revision was not incorporated into the tracked codebase; software info still showed only two code files.

ChatDev: Pong

Task. Develop a two-player Pong game. Each player controls a vertical paddle, moving up and down to bounce a ball back. The ball bounces between the two paddles and off the top and bottom edges. Players score a point when the opponent misses the ball. Incorporate a winning score threshold and reset the ball after each point.

Outcome. Original GPT-4o run: solved (human annotation). Reproduction: solved (judge). Screened cat-2 prior: control.

MAST modes (judge). 1.1, 1.3, 1.4, 2.1, 2.4, 2.5, 2.6, 3.1, 3.2, 3.3

Run narrative (judge, tier 1). The run selected an Application modality and Python, generated a multi-file Pygame Pong implementation, then entered review cycles where reviewers repeatedly identified a real ball-position/collision/scoring issue. Attempts to modify the code were truncated and apparently not cleanly applied; subsequent review/test phases often saw stale or even internally corrupted versions of the code rather than the latest attempted fixes. Testing first failed because pygame was missing, the system installed it externally, then a programmer attempted an unnecessary/incomplete pygame shim even though the test passed once the dependency was installed. The final artifacts included code, requirements, and a manual, but information flow was inconsistent: code-review fixes were lost, requirements were not passed into the manual phase, and tests only established that the program could start after installing pygame, not that gameplay worked correctly.

Failure summary (judge, tier 2). The system produced a runnable Pong implementation, but suffered from instruction noncompliance, repeated stale reviews, lost/unpropagated fixes and requirements, ignored reviewer feedback, truncated/corrupted modification attempts, and shallow/incorrect verification.

Mechanisms flagged.

- *missing dependency*: The first code version depended on pygame but no environment file existed yet, leading directly to a test failure later.
- *physics bug*: The generated ball update logic checked top/bottom wall collisions using the old rect before syncing the rect to the new floating-point position.
- *truncated output*: The first code-review modification response was truncated mid-file, producing incomplete code.
- *state corruption*: After the truncated modification, the system continued with stale or inconsistent code state rather than a cleanly modified version.
- *code corruption*: A later placeholder contained severely corrupted source code that did not match the originally generated code.
- *stale review*: Despite the attempted modifications, later review phases repeatedly reviewed the original stale code with the same defect.

ChatDev: Strands

Task. Implement a version of the NYT Strands puzzle. Strands is a word search game within a 6x8 grid of letters which players use to uncover words falling under a theme. The board must be filled entirely to complete the puzzle - no words will overlap. Each game includes a challenge word called the "spangram" that touches two opposite sides of the board. Themed words will highlight blue; the spangram will highlight yellow. Words can be formed by connecting adjacent letters in any direction (vertically, horizontally, diagonally) and can change direction mid-word. Players can find non-theme words to earn hints. Every 3 non-theme words will unlock a hint! The puzzle must be fully completed by finding all theme words and filling the board.

Outcome. Original GPT-4o run: failed (human annotation). Reproduction: failed (judge). Screened cat-2 prior: medium.

MAST modes (judge). 1.1, 1.3, 1.4, 2.2, 2.4, 2.5, 2.6, 3.1, 3.2, 3.3

Run narrative (judge, tier 1). The run chose an Application modality and Python, then the Programmer attempted a multi-file Tkinter implementation of a Strands-like game. The initial coding response was incomplete: it referenced missing UI modules and emitted a truncated board module, while also embedding inconsistent puzzle data. The Code Reviewer caught several serious issues, but also introduced or overstated some claims, and the subsequent modification phase produced only a partial response at first; later code state became corrupted by markdown/language labels being inserted as literal ‘python‘ lines in source files. The final codebase had five files and a manual, but the generated sample puzzle was internally invalid, the spangram path did not spell the spangram or touch opposite sides, hints/non-theme play were effectively absent, and the literal ‘python‘ lines would prevent execution. Environment and manual phases proceeded despite these defects, with requirements information partly lost and the manual describing features that the final code did not actually support.

Failure summary (judge, tier 2). The system produced a broken Strands application after incomplete code generation, state/artifact loss, ignored review fixes, markdown leakage into source files, and no effective final verification.

Mechanisms flagged.

- *truncated code*: The initial coding response claimed to provide a complete implementation, but the trace shows the ‘strands_board.py‘ code block cut off mid-line.
- *missing modules*: The initial Programmer referenced modules/classes that were not saved as code files.
- *dropped files*: The code updater saved only three code files after the first coding phase, despite the Programmer listing more components.
- *bad puzzle data*: The initial sample puzzle data did not match its word definitions: ‘abracadabra‘ was mapped to only eight cells on a row spelling ‘ABRACDRA‘.
- *missing requirement*: The initial puzzle completion condition did not require full-board coverage even though the user explicitly required the board to be filled entirely.
- *unreachable action*: The initial UI defined ‘submit_word()‘ but no visible submit control was wired to it.

ChatDev: Sudoku

Task. Develop a classic Sudoku puzzle game that uses a 9x9 grid. Each row, column, and 3x3 subgrid must contain the digits 1 through 9 exactly once. The program should allow the player to input values for specific cells, check for mistakes, and confirm when the puzzle is completed.

Outcome. Original GPT-4o run: failed (human annotation). Reproduction: failed (judge). Screened cat-2 prior: high.

MAST modes (judge). 1.1, 1.3, 1.5, 2.4, 2.5, 3.1, 3.2

Run narrative (judge, tier 1). The run chose an Application modality and Python, then initially generated a fairly complete Tkinter GUI Sudoku implementation, but the code-extraction/update step badly misparsed the response and wrote only ‘python main.py‘ into ‘sudoku_game.py‘. The reviewer correctly detected that fatal extraction failure, but the subsequent repair replaced the GUI with a command-line Sudoku, dropping the GUI requirement that had been explicitly injected by the system. Later reviews identified that move validation was not actually used and completion was tied to a hard-coded solution, but the modification attempts were truncated and/or not applied; meanwhile internal code representations became inconsistent and sometimes visibly corrupted. Despite these unresolved issues, the system reported “Test Pass,” generated an empty/no-dependency requirements file through a redundant reflection step, and wrote a manual that overstated validation behavior and ended abruptly.

Failure summary (judge, tier 2). The system lost the original GUI Sudoku implementation during code extraction, repaired it as a CLI despite the GUI requirement, ignored later validation-review fixes, and then passed the project with only superficial testing.

Mechanisms flagged.

- *gui requirement*: The Coding phase explicitly required a GUI because gui_design was enabled, and the initial programmer complied with a Tkinter GUI.
- *format violation*: The Programmer's initial response included explanatory sections, run instructions, and future-offer text beyond the strict file-code-block format, which may have contributed to extraction failure.
- *code extraction*: The code update step catastrophically mis-extracted the initial 'sudoku_game.py', replacing the full GUI implementation with a shell command.
- *file corruption*: The extracted 'sudoku_game.py' content was literally 'python main.py' rather than Python source code.
- *metric symptom*: Software metrics immediately reflected the extraction failure: only 11 code lines were counted despite the initial response containing a large implementation.
- *review catch*: The Code Reviewer correctly identified the fatal missing implementation and explained that 'SudokuGame' was undefined.

ChatDev: TextBasedSpaceInvaders

Task. Program a simplified Space Invaders game. The player controls a ship at the bottom of the screen and can move horizontally and fire shots to destroy descending alien rows. The game ends if aliens reach the bottom or the player defeats all aliens. Add multiple alien rows, a limited number of lives, and score tracking.

Outcome. Original GPT-4o run: failed (human annotation). Reproduction: solved (judge). Screened cat-2 prior: high.

MAST modes (judge). 1.1, 1.3, 2.1, 2.4, 2.5, 2.6, 3.1, 3.2

Run narrative (judge, tier 1). The run selected an Application modality and Python, then the Programmer attempted a multi-file Pygame Space Invaders implementation, but the first coding response was truncated before completing game.py and the code parser only saved five files, leaving main.py importing a missing Game class. Code review correctly identified the missing game.py and a missing YELLOW import, but successive modification attempts were also truncated or not parsed into game.py, causing repeated reviews and eventually a failed test where Python imported an unrelated installed package named game. The test-summary phase diagnosed the missing local game.py, and the test-modification phase finally added game.py; a smoke test then passed. The run generated requirements.txt and a manual afterward, but dependency information was not carried into the manual phase, and the final manual response appears truncated mid-controls section.

Failure summary (judge, tier 2). Despite final smoke-test success, the system repeatedly failed to persist and apply the missing game.py fix, causing repeated reviews/import failures, with only weak verification and a truncated manual.

Mechanisms flagged.

- *truncated code*: The first coding response claimed to provide a complete runnable implementation, but game.py was cut off mid-statement.
- *lost file*: Although the Programmer described six files including game.py, the code update stage only saved five code files and did not create game.py.

- *missing import*: The initial player.py used YELLOW in shoot() without importing it.
- *state divergence*: The first Code Reviewer correctly diagnosed the persisted state rather than the Programmer's intended full answer: game.py was missing and player.py lacked YELLOW.
- *truncated fix*: The first CodeReviewModification attempted to add game.py but was truncated before completing even the event handler.
- *partial application*: After the first modification, only player.py and alien.py were updated; the missing game.py was not added despite being the main requested fix.

ChatDev: The Crossword

Task. Implement a crossword puzzle. Provide a grid of squares with clues for across and down entries. The user can enter words by specifying the clue number and direction. The application validates if letters match and confirms completion when all correct words are filled in.

Outcome. Original GPT-4o run: failed (human annotation). Reproduction: failed (judge). Screened cat-2 prior: low.

MAST modes (judge). 1.1, 1.3, 1.4, 2.1, 2.4, 2.5, 3.1, 3.2, 3.3

Run narrative (judge, tier 1). The run selected an Application modality and Python quickly, then generated a three-file Tkinter crossword app with a grid, clues, clue-number/direction entry, validation, and completion detection. The first code review correctly identified that the sample crossword was internally inconsistent and likely unsolvable, but the subsequent modification responses were truncated/incomplete and the review loop repeatedly re-reviewed essentially the original code rather than successfully applying fixes. Internal placeholders also showed corrupted versions of the code that differed from what agents saw, and despite three review cycles still producing high-priority comments, the system proceeded to testing, which reported success without catching the unresolved logical defect. Requirements were generated redundantly and then not passed into the manual phase, and the final manual output was truncated mid-section; overall, the project produced files, but the crossword puzzle remained logically flawed and documentation incomplete.

Failure summary (judge, tier 2). The system produced and delivered a crossword app with an invalid/unfixed puzzle and truncated manual after repeated review cycles, ignored reviewer fixes, stale/corrupted code flow, and superficial testing that incorrectly passed the artifact.

Mechanisms flagged.

- *invalid puzzle*: The initial sample crossword contained conflicting overlapping answers, making at least several entries impossible to validate correctly.
- *missing validation*: The model built the solution grid by blindly overwriting cells and did not check for clue conflicts or bounds during construction.
- *unresolved review*: The first code review correctly identified the invalid crossword as the highest-priority problem.
- *truncated patch*: The first attempted code-review modification was truncated in the middle of ‘crossword_model.py‘.

- *state not updated*: After the truncated modification, later review prompts still showed the original invalid puzzle rather than a successfully modified version.
- *code corruption*: Internal placeholders in later review cycles contained corrupted code that differed from the displayed code, including lost underscores and broken event binding.

ChatDev: TicTacToe (with display)

Task. Design a tic-tac-toe game with a user-friendly interface, allowing two players to take turns and determining the winner. Use a standard 3x3 grid, track each player's moves. Players alternate placing X or O, and the game ends when a player wins or the board is full.

Outcome. Original GPT-4o run: failed (human annotation). Reproduction: failed (judge). Screened cat-2 prior: medium.

MAST modes (judge). 1.1, 1.3, 1.4, 2.1, 2.2, 2.4, 2.5, 3.1, 3.2, 3.3

Run narrative (judge, tier 1). The run began by choosing a website modality for the tic-tac-toe task, but the subsequent language-selection phase overrode the earlier HTML/CSS/JavaScript direction by selecting Python, leading the programmer to build a Flask-backed web app. The initial implementation mostly covered the requested game logic and UI, but the generated HTML/CSS/JS inherited Python-style triple-quoted docstrings and the file-path information for Flask templates/static assets was not reliably preserved. Code review correctly identified these issues, but multiple review/modification cycles repeated the same comments while modification outputs became truncated or corrupted, and state appeared to revert or diverge between phases. Despite unresolved front-end syntax/path issues and incomplete generated code snippets, the test phase reported success. The environment dependency was identified as Flask, but the requirements information was not cleanly passed into the manual phase, and the final manual itself was cut off before the usage instructions were completed.

Failure summary (judge, tier 2). The system repeatedly failed to incorporate review fixes for invalid Flask asset structure and non-Python triple-quoted front-end files, accepted truncated/corrupted code after superficial testing, and ended with an incomplete manual.

Mechanisms flagged.

- *invalid asset syntax*: The generic code-block/docstring format caused Python triple-quoted strings to be placed at the tops of HTML, CSS, and JavaScript files.
- *path loss*: The programmer planned Flask folder paths, but the update phase appeared to create/update flat files rather than preserving 'templates/' and 'static/' paths.
- *escaping distortion*: Code passed through placeholders became HTML-escaped, distorting operators and arrow syntax in downstream context.
- *unresolved review*: The first code review correctly identified file-placement and triple-quote/front-end issues, but the requested modifications did not successfully resolve them.
- *truncated modification*: The first code-review modification output was truncated in the middle of 'game.py'.
- *state corruption*: Between review cycles, the code state became severely corrupted in placeholders, including broken Flask initialization, missing dunder methods, stripped HTML, and malformed JavaScript.

ChatDev: Tiny Rouge

Task. Design a roguelike game inspired by Tower of the Sorcerer. Use a fixed 80x80 grid map. The player character is controlled using W/A/S/D keys for movement (up, left, down, right). The character can move only on floor tiles and cannot pass through walls. The goal is to reach the door to proceed to the next level. The player encounters monsters; combat is resolved by subtracting the monster's HP from the player's HP. HP is restored by 20–30 points when the player touches a treasure chest. Ensure there is always at least one valid path from the starting position to the door. Include a minimal UI to display the player's current HP and encountered monster stats.

Outcome. Original GPT-4o run: failed (human annotation). Reproduction: failed (judge). Screened cat-2 prior: high.

MAST modes (judge). 1.1, 1.3, 1.4, 1.5, 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, 3.1, 3.2, 3.3

Run narrative (judge, tier 1). The run selected an Application modality and Python, then the Programmer attempted a Pygame roguelike implementation. The first coding output was cut off inside ‘game.py‘, so only four files were saved even though ‘main.py‘ imported the missing ‘game‘ module; subsequent “code completion” cycles repeatedly and incorrectly focused on ‘entities.py‘ before an empty/unhelpful completion prompt caused the Programmer to reconstruct ‘game.py‘. Review then raised a door/path issue, partially based on a questionable reading of the code, and the modification phase added explicit door walkability plus other gameplay refinements. A superficial test reported success, then dependency and manual phases ran, but the requirements information was inconsistently wrapped, reflected, and ultimately appears not to have flowed into the manual phase. The final artifact likely runs and implements the requested core mechanics, but the trace shows missing/intermediate broken state, repeated redundant work, questionable review/test coverage, UI overlap with the map, and documentation/dependency handoff problems.

Failure summary (judge, tier 2). The system eventually produced a runnable-looking roguelike, but the workflow suffered from truncated/incomplete code, repeated and misdirected completion cycles, lost dependency information, shallow verification, and final delivery artifacts that were not cleanly saved or fully validated.

Mechanisms flagged.

- *truncated code*: The Programmer claimed to provide a complete functional implementation, but the initial ‘game.py‘ output was truncated mid-function.
- *missing file*: Because the initial ‘game.py‘ code block was incomplete, the code updater saved only four files even though ‘main.py‘ imports ‘game‘.
- *false incomplete*: The CodeComplete phase misidentified ‘entities.py‘ as an unimplemented file, apparently because ‘Door‘ contained ‘pass‘, even though a dataclass marker class with ‘pass‘ was already valid and intentional.
- *redundant work*: The Programmer responded to the first CodeComplete request by reproducing essentially the same ‘entities.py‘, leaving the real missing-file problem untouched.
- *repeated cycle*: The same ‘entities.py‘ completion was requested and answered a second time with the same content.
- *empty target*: The third CodeComplete prompt had an empty unimplemented file, producing nonsensical instructions, but the Programmer inferred and created ‘game.py‘ anyway.

ChatDev: Wordle

Task. Design a standard wordle game by providing a daily 5-letter English language word. The game should allow a player to take 6 turns to guess the word. Correct guess for each letter is colored by green (correct letter in correct position), grey for invalid letter, or yellow for correct letter in wrong position. It should be playable from Linux Terminal, and does not require me to access a dedicated UX or GUI platform, i.e. print the results on terminal at each stage and let me play it there by entering inputs. If the player guesses the word within the allowed number of attempts, they win. If not, the solution is revealed. Validate each guess for proper feedback.

Outcome. Original GPT-4o run: failed (human annotation). Reproduction: failed (judge). Screened cat-2 prior: medium.

MAST modes (judge). 1.1, 1.3, 1.4, 2.2, 2.4, 2.5, 2.6, 3.1, 3.2, 3.3

Run narrative (judge, tier 1). The run selected an Application/Python approach for a terminal Wordle game, then the coding phase produced a multi-file Python implementation despite a contradictory GUI instruction injected by configuration. The initial code mostly matched the requested terminal game, but contained a limited word list including at least one six-letter word, and the review cycle introduced churn: the reviewer first falsely flagged already-correct duplicate-letter logic, the programmer changed validation to call a nonexistent function, and a later review caught the import mismatch. Several modification outputs were visibly truncated or only partially applied, and some internal code placeholders became corrupted between phases. Automated testing then ran the interactive program without stdin, got EOFError, and the system "fixed" this by making the program exit immediately in non-interactive contexts; the next test passed only because the game no longer reached gameplay under that harness. Requirements and manual phases followed, but the generated requirements content appears not to have propagated into the manual phase, and the final manual output is visibly cut off mid-sentence. The final project therefore has useful code, but the run's validation was superficial and some user-facing/documentation and latent correctness issues remain.

Failure summary (judge, tier 2). The system produced a terminal Wordle app but left latent task violations and incomplete artifacts while relying on superficial testing and losing/ignoring key review and dependency information.

Mechanisms flagged.

- *config conflict*: The global configuration enabled GUI design even though the user explicitly requested a Linux terminal game with no dedicated GUI platform.
- *prompt contradiction*: The Coding prompt injected a GUI requirement that contradicted both the user request and the earlier modality rationale.
- *invalid word list*: The initial word list included a non-5-letter entry, violating the core daily 5-letter requirement and creating a latent runtime failure if selected as the daily solution.
- *false review*: The first Code Reviewer falsely claimed the duplicate-letter scoring code had a bug and possible KeyError, even though the shown code already checked the key with 'solution_counts.get(g, 0) > 0' before decrementing 'solution_counts[g]'.
- *multi-issue review*: The first review mixed its highest-priority duplicate-letter claim with a separate validation-policy concern, even though it was asked to propose one highest-priority comment.

- *introduced bug*: The Programmer attempted to address the validation comment by importing and calling ‘is_allowed_guess‘, but did not successfully add that function to ‘word_bank.py‘.

B Case briefs: Magentic-One (GAIA)

Magentic-One 00d579ea (GAIA L3)

Task. Assuming scientists in the famous youtube video The Thinking Machine (Artificial Intelligence in the 1960s) were interviewed the same year, what is the name of the scientist predicting the sooner thinking machines or robots? Answer using the format First name Last name

Outcome. Original GPT-4o run: failed. Reproduction: failed (normalized exact match; expected answer: "Claude Shannon"). Screened cat-2 prior: medium.

MAST modes (judge). 1.1, 1.3, 1.5, 2.1, 2.2, 2.3, 2.5, 2.6, 3.1, 3.2, 3.3

Run narrative (judge, tier 1). The run began with the orchestrator assigning WebSurfer to find the YouTube video and transcript for "The Thinking Machine (Artificial Intelligence in the 1960s)," but the process quickly became dominated by search-result snippets and repeated planning rather than extraction of the actual video content. WebSurfer initially found a Bing result whose snippet named Jerome Wiesner, Oliver Selfridge, and Claude Shannon, but clicking it opened a Bing Videos page rather than the YouTube watch page, and several subsequent attempts repeated the same action. The orchestrator repeatedly summarized the same uncertainty, elevated Oliver Selfridge as a hunch, and issued near-identical "new plans" insisting that no answer should be finalized without transcript or speaker-label evidence. Eventually WebSurfer opened the YouTube watch page and confirmed only the description and metadata, but did not inspect captions, transcript controls, comments, timestamps, or the video itself; later searches hit Bing human-verification pages, after which the orchestrator continued looping through self-diagnosis. Despite repeatedly stating that the answer remained unverified and should not be guessed from reputation, the orchestrator ended with "Oliver Selfridge," a requested-format answer but one not supported by direct quote-to-speaker evidence from the trace.

Failure summary (judge, tier 2). The system repeatedly looped on search/video-discovery steps, ignored its own/direct-source instructions and evidence threshold, then prematurely returned an unsupported and misformatted guess without transcript or speaker verification.

Mechanisms flagged.

- *wrong page*: When asked to open the YouTube result, WebSurfer opened or remained on Bing Videos rather than the actual YouTube watch page.
- *repeated no-op*: WebSurfer repeated the same click/action on the Bing Videos page multiple times without making progress.
- *hunch reinforcement*: The orchestrator repeatedly expanded the same fact sheet and plan, adding lengthy hunches without new evidence.
- *caution mismatch*: The orchestrator kept stating that transcript verification was still missing, but simultaneously kept strengthening a preferred candidate.
- *instruction drift*: When instructed to open the direct YouTube page and avoid Bing, WebSurfer instead typed another Bing search query and immediately encountered a verification challenge.
- *blocked loop*: Bing human verification blocked repeated searches, but the system continued trying similar Bing searches several times.

Magentic-One 023e9d44 (GAIA L2)

Task. It's May 2023, and I'm about to drive across the U.S. from California to Maine. I always recycle my water bottles at the end of a trip, and I drink 5 12-ounce water bottles for every 100 miles I travel, rounded to the nearest 100. Assuming I follow I-40 from Los Angeles to Cincinnati, then take I-90 from Cincinnati to Augusta, how many dollars will I get back according to Wikipedia?

Outcome. Original GPT-4o run: failed. Reproduction: failed (normalized exact match; expected answer: "8"). Screened cat-2 prior: medium.

MAST modes (judge). 1.1, 1.3, 1.5, 2.2, 2.5, 2.6, 3.1, 3.2, 3.3

Run narrative (judge, tier 1). The run began with the orchestrator decomposing the user's trip/refund word problem into distance lookup, Wikipedia deposit lookup, and arithmetic, but it immediately seeded several unverified hunches, especially that Maine's deposit was likely 5?. WebSurfer first searched Bing and surfaced only a Bing snippet from Maine government/non-Wikipedia sources, then repeatedly tried Bing searches for route mileages despite Bing returning a human-verification challenge and despite the orchestrator repeatedly instructing it to bypass Bing and use direct sources. The orchestrator then spent many turns restating updated fact sheets and plans, increasingly promoting the guessed 4,000-mile/$10 result while still noting that neither distances nor the Wikipedia deposit had been verified. Eventually WebSurfer opened the Maine Wikipedia page directly, but only the top of the page was visible and no bottle-deposit amount was found; nevertheless, the orchestrator immediately ended with "FINAL ANSWER: 10," without sourced mileages, without a Wikipedia quote for the deposit, and without using the planned terminal arithmetic.

Failure summary (judge, tier 2). The system repeatedly ignored instructions to avoid Bing, failed to verify the route mileages and Wikipedia deposit value, then prematurely returned a guessed $10 answer.

Mechanisms flagged.

- *premature hunch*: The initial fact sheet introduced an unverified guess that the answer likely depends on Maine's 5? deposit before any Wikipedia source was opened.
- *indirect source*: WebSurfer's first lookup used Bing rather than going directly to Wikipedia, and the visible result was a search page/snippet, not a Wikipedia citation.
- *source drift*: The only early deposit text visible came from Maine government and other non-Wikipedia sources, but it was later treated as supporting the Wikipedia answer.
- *ignored instruction*: Despite being told to switch away from Bing, WebSurfer kept using browser search queries that opened Bing challenge pages.
- *repeated search*: WebSurfer explicitly continued to type search queries into the browser search bar after being asked for non-Bing/direct sources.
- *planning loop*: The orchestrator repeatedly restated plans and fact sheets instead of changing tactics, invoking other agents, or providing concrete direct URLs.

Magentic-One 0383a3ee (GAIA L1)

Task. On the BBC Earth YouTube video of the Top 5 Silliest Animal Moments, what species of bird is featured?

Outcome. Original GPT-4o run: solved. Reproduction: solved (normalized exact match; expected answer: "Rockhopper penguin"). Screened cat-2 prior: control.

MAST modes (judge). 1.1, 1.3, 1.4, 2.3, 2.4, 2.5, 2.6, 3.1, 3.2, 3.3

Run narrative (judge, tier 1). The run began with model-mismatch warnings and an orchestrator-generated plan to find the BBC Earth YouTube video and identify the featured bird species, but WebSurfer repeatedly used Bing and hit human-verification challenges even after being told not to. WebSurfer eventually reached the official BBC Earth YouTube channel, then YouTube search, found and opened the correct video page, where the title, description, and visible chapter text established that the bird segment involved penguins, especially the chapter "HILARIOUS PENGUINS LEARN TO CLIMB." However, the agents did not properly inspect the description, transcript menu, chapters, or video frames for species-level evidence; WebSurfer scrolled into comments and clicked the captions button instead of opening a transcript or scrubbing frames. Later, the correct video URL was silently lost and replaced by an unrelated unavailable YouTube URL, which the orchestrator incorrectly treated as the target video becoming unavailable. The system then returned to blocked Bing searches despite repeated instructions to avoid Bing and finally answered "rockhopper penguin" based on an inferred hunch from rocky penguin behavior rather than verified text or visual evidence from the source.

Failure summary (judge, tier 2). The system likely produced the correct species, but repeatedly ignored non-Bing/direct-video instructions, lost the known correct YouTube URL, performed weak or incorrect verification, and ended with an uncaveated inferred answer.

Mechanisms flagged.

- *instruction ignored*: After the orchestrator explicitly told WebSurfer to avoid Bing and go directly to YouTube or the BBC Earth channel, WebSurfer again typed a query into the browser search bar and landed on Bing.
- *route regression*: Even after being instructed to use YouTube directly, WebSurfer entered a site:youtube.com search that routed to Bing again.
- *state loss*: After opening the correct video, the page metadata and description were available, including the correct video ID, but the team did not preserve that URL robustly for later use.
- *wrong page area*: Instead of opening the description, transcript, chapters, or video controls, WebSurfer scrolled into comments.
- *transcript mishandled*: WebSurfer clicked the captions/CC control and treated it as the transcript/captions path, but did not open YouTube's transcript or description controls.
- *action mismatch*: When asked to scrub through the video and inspect frames, WebSurfer clicked the captions shortcut again rather than scrubbing or pausing on useful frames.

Magentic-One 04a04a9b (GAIA L2)

Task. If we assume all articles published by Nature in 2020 (articles, only, not book reviews/columns, etc) relied on statistical significance to justify their findings and they on average came to a p-value of 0.04, how many papers would be incorrect as to their claims of statistical significance? Round the value up to the next integer.

Outcome. Original GPT-4o run: failed. Reproduction: failed (normalized exact match; expected answer: “41”). Screened cat-2 prior: low.

MAST modes (judge). 1.1, 1.3, 2.2, 2.5, 3.1, 3.2

Run narrative (judge, tier 1). The system began by decomposing the task into two parts: find Nature’s 2020 article-only count and interpret the p-value assumption. It initially tried to use Bing through WebSurfer, repeatedly hit a human-verification challenge, and despite the orchestrator saying to avoid Bing and go directly to Nature, WebSurfer continued using Bing search for several turns. Eventually WebSurfer navigated directly to Nature’s 2020 articles page, opened the Article Type dropdown, and surfaced the relevant count, ”Article (1002)”. The count was passed to Assistant, which first produced an unjustified proportional calculation yielding 802; the orchestrator noticed this and asked for a more careful interpretation. Assistant then concluded that because 0.04 is below 0.05, zero papers would be incorrect, and the orchestrator finalized with only ”0”. The run did obtain a plausible article count, but the statistical interpretation remained under-justified: an average p-value below 0.05 does not by itself determine how many individual papers were or were not statistically significant, and the final answer dropped the ambiguity, source, and reasoning.

Failure summary (judge, tier 2). The system eventually found the Nature article count but failed to resolve the ambiguous statistical interpretation, repeated blocked Bing-search behavior despite direct-navigation instructions, and finalized an unsupported answer of 0 after weak verification.

Mechanisms flagged.

- *unresolved ambiguity*: The initial fact sheet identified the key statistical interpretation as ambiguous but did not resolve it before proceeding.
- *instruction drift*: After being asked to open Nature’s archive directly, WebSurfer still used the browser search bar and landed on Bing again.
- *repeated failure*: The same blocked Bing-search pattern was repeated multiple times with slightly varied queries.
- *lost instruction*: The orchestrator recognized that Bing was unusable but then the subsequent WebSurfer action still used Bing.
- *bad calculation*: Assistant’s first calculation was mathematically and conceptually unjustified.
- *average misuse*: Assistant’s second calculation assumed that an average p-value of 0.04 means no paper could be incorrect about statistical significance.

Magentic-One 05407167 (GAIA L2)

Task. In the 2018 VSCode blog post on replit.com, what was the command they clicked on in the last video to remove extra lines?

Outcome. Original GPT-4o run: failed. Reproduction: failed (normalized exact match; expected answer: “Format Document”). Screened cat-2 prior: medium.

MAST modes (judge). 1.1, 1.3, 2.3, 2.5, 2.6, 3.1, 3.2, 3.3

Run narrative (judge, tier 1). The run began with a reasonable plan to find a 2018 Replit blog post about VSCode and inspect its last embedded video, but it immediately tried Bing and hit

a human-verification challenge. The orchestrator redirected WebSurfer to replit.com/blog, where WebSurfer opened the modern blog homepage and repeatedly typed or input "VSCode" without any visible search results, scrolling, filtering, article opening, or video inspection. The orchestrator then entered a long loop of self-diagnosis and updated fact sheets, repeatedly noting that the exact 2018 post title and the command were still unverified while also amplifying guesses such as "Remove Empty Lines." Despite repeatedly instructing WebSurfer to avoid Bing, the system later returned to Bing and hit the same challenge again. No agent ever identified the article title/date, opened a candidate 2018 post, inspected the last video, read captions, or saw the command on screen; nevertheless the orchestrator ended with "FINAL ANSWER: Remove Empty Lines," apparently derived from its earlier hunch rather than source evidence.

Failure summary (judge, tier 2). The system repeatedly stalled in search/planning loops, ignored instructions to avoid Bing, never identified the 2018 post or inspected the last video, and then prematurely returned an unverified guessed command.

Mechanisms flagged.

- *stalled browsing*: WebSurfer opened the Replit blog homepage but only reported the top of a 2026-era page, not any 2018 search results.
- *ineffective input*: WebSurfer repeatedly claimed to input "VSCode," but the visible page content stayed unchanged and showed no search results.
- *instruction loop*: The orchestrator repeated nearly the same request to WebSurfer multiple times without changing tactics or decomposing the browser interaction more concretely.
- *overstated fact*: The orchestrator upgraded weak evidence into a stated verified fact that the blog page was searchable/filterable and that the search field contained "VSCode," even though the visible viewport did not show results or a populated search field.
- *state churn*: The orchestrator repeatedly summarized the same missing facts instead of taking new action through another agent or tool.
- *unacted diagnosis*: The orchestrator explicitly diagnosed the root cause as stalled discovery, but then continued producing more plans and fact sheets rather than changing execution strategy.

Magentic-One 08cae58d (GAIA L2)

Task. According to Google Finance, when was the first year the Apple stock went above $50 (without adjusting for stock split)?

Outcome. Original GPT-4o run: failed. Reproduction: failed (normalized exact match; expected answer: "2018"). Screened cat-2 prior: medium.

MAST modes (judge). 1.1, 2.5, 2.6, 3.1, 3.2, 3.3

Run narrative (judge, tier 1). The run began with repeated model-mapping warnings and an orchestrator fact sheet that already guessed the answer was 1987 before any lookup. WebSurfer first searched via Bing and hit a challenge, then navigated directly to Google Finance's AAPL page, clicked the MAX chart, and hovered once, producing a tooltip for Oct 23, 1987 with a displayed price of 0.32. No historical table, split-adjustment label, or actual first crossing above $50 was found. Despite this, the orchestrator asserted that Google Finance values were split-adjusted and

that 1987 was the first unadjusted year above $50, telling Assistant to answer accordingly. Assistant repeated those assertions as evidence, including a distorted claim that the Oct 23, 1987 tooltip was near or indicated the $50 crossing, and the final answer was simply 1987.

Failure summary (judge, tier 2). The system prematurely produced an unsupported final answer by ignoring the only tooltip value obtained from Google Finance and failing to verify split-adjustment labeling or the first $50 crossing.

Mechanisms flagged.

- *anchoring bias*: The orchestrator seeded the work with a strong answer guess before evidence was gathered, creating anchoring risk.
- *unused requirements*: The initial fact sheet recognized several needed facts, including exact historical data and interpretation of split adjustment, but these were never actually established.
- *incomplete inspection*: The orchestrator asked for historical data or MAX chart details, but WebSurfer only clicked MAX and did not provide any actual earliest-crossing analysis.
- *vague interaction*: When asked again to inspect the chart, WebSurfer performed a vague hover action rather than systematically sampling or extracting data around the threshold.
- *contradictory evidence*: The only tooltip data obtained showed a price of 0.32 on Oct 23, 1987, which does not itself show Apple above $50.
- *missing label*: No text indicating whether Google Finance prices were split-adjusted or unadjusted was captured, despite repeated requests for exactly that.

Magentic-One 27d5d136 (GAIA L1)

Task. $\neg(A \land B) \leftrightarrow (\neg A \lor \neg B)$ $\neg(A \lor B) \leftrightarrow (\neg A \land \neg B)$ $(A \to B) \leftrightarrow (\neg B \to \neg A)$ $(A \to B) \leftrightarrow (\neg A \lor B)$ $(\neg A \to B) \leftrightarrow (A \lor \neg B)$ $\neg(A \to B) \leftrightarrow (A \land \neg B)$

Which of the above is not logically equivalent to the rest? Provide the full statement that doesn't fit.

Outcome. Original GPT-4o run: solved. Reproduction: solved (normalized exact match; expected answer: "$(\neg A \to B) \leftrightarrow (A \lor \neg B)$"). Screened cat-2 prior: control.

MAST modes (judge). 2.6

Run narrative (judge, tier 1). The user asked which of six propositional-logic biconditional statements was not logically equivalent to the rest. The orchestrator generated an initial fact sheet that already identified the likely odd statement, correctly noting that $\neg A \to B$ rewrites as $A \lor B$ rather than $A \lor \neg B$. Despite assembling a multi-agent team and stating a plan to use the Assistant, no specialist agent was actually invoked; the orchestrator immediately produced the final answer itself. The final answer gave only the suspect full statement, without explanation or verification, but it appears to satisfy the user's requested output and is logically correct.

Failure summary (judge, tier 2). The task was answered correctly, but the orchestrator did not follow its own stated inter-agent plan and answered directly instead of using the Assistant.

Mechanisms flagged.

- *plan not followed*: The plan said to use the Assistant to present the final reasoning, but the Assistant never appears in the trace after team assembly.

Magentic-One 2b3ef98c (GAIA L2)

Task. Could you help me out with this assignment? Our professor sprung it on us at the end of class Friday, and I'm still trying to figure it out. The question he asked us was about an anagram. I've attached an audio recording of the question that he asked, so if you could please take a listen and give me the answer, I'd really appreciate the help. Please limit your response to the anagram text that could be generated from the original line which fulfills the professor's request, without any other commentary. Also, please don't include any punctuation in your response.

Outcome. Original GPT-4o run: failed. Reproduction: failed (normalized exact match; expected answer: "To be or not to be that is the question whether tis nobler in the mind to suffer the slings and arrows of outrageous fortune"). Screened cat-2 prior: low.

MAST modes (judge). 1.1, 1.2, 1.3, 1.5, 2.1, 2.2, 2.3, 2.5, 2.6, 3.1, 3.2, 3.3

Run narrative (judge, tier 1). The run began with a model-mapping warning and an orchestrator plan to inspect an MP3, transcribe it, solve an anagram, and return only the required text. FileSurfer quickly produced a transcript saying the professor had supplied a long anagram and asked students to recover the original Shakespeare line, but it did not provide the requested metadata. The orchestrator and Assistant immediately inferred the famous Hamlet phrase "To be or not to be that is the question," repeatedly returned variants of it, and then entered several self-correcting loops about whether the task wanted the original line or an anagram and whether the file had been properly verified. Subsequent FileSurfer calls repeated the same transcript or even returned only the inferred answer, while a wrong absolute path caused a file error, a terminal request initially lacked a code block, and a later terminal inspection produced no output. The system never conclusively verified metadata, letter counts, or whether the much longer spoken anagram corresponded to a longer Shakespeare line, and the run ultimately crashed with a ledger parsing error before delivering a final user-facing answer.

Failure summary (judge, tier 2). The orchestrator conflated the anagram/original-line requirement, repeated answer and inspection loops, ignored tool/path feedback, performed weak or failed verification, and crashed before delivering a final user-facing answer.

Mechanisms flagged.

- *dropped metadata*: The orchestrator requested metadata, duration, codec details, and transcript-like clues, but FileSurfer returned only a transcript and no technical metadata.
- *prompt ambiguity*: The transcript itself distinguished between the supplied anagram and the requested original line, but that distinction became unstable later.
- *premature inference*: The Assistant jumped directly to the short Hamlet quotation without verifying that it matched the long anagram from the transcript.
- *instruction conflation*: The orchestrator asked for "the anagram text" even though the transcript said the professor wanted the original line.
- *repeated work*: The system repeated the same candidate answer several times instead of advancing verification or finalizing cleanly.

- *redundant inspection*: The same FileSurfer inspection was requested again, and FileSurfer again returned the same transcript rather than new metadata.

Magentic-One 366e2f2b (GAIA L2)

Task. The attached file lists accommodations in the resort town of Seahorse Island. Based on the information in this file, which seems like the better available place to stay for a family that enjoys swimming and wants a full house?

Outcome. Original GPT-4o run: failed. Reproduction: failed (normalized exact match; expected answer: "Shelley's place"). Screened cat-2 prior: medium.

MAST modes (judge). 1.1, 2.2, 2.4, 2.5, 3.1, 3.2, 3.3

Run narrative (judge, tier 1). The system received a PDF-based accommodation-selection task, built a plan to inspect the file, and asked FileSurfer to extract availability, full-house/rental-house status, and swimming-related amenities. FileSurfer opened the one-page PDF but returned a flattened table rather than a row-aligned summary, so the names, ratings/vacancies, pool values, and reviews were visually separated and required careful reconstruction. The orchestrator then asked the Assistant to choose the best available rental house, but it did not repair or verify the table alignment. The Assistant selected Headrush Beach and asserted that it was available and had a pool, even though reconstructing the extracted rows in order indicates Headrush Beach is a rental house with vacancy "No" and pool "No"; the available rental house with a pool appears to be Shelley's Place. The final answer further compressed the Assistant's rationale down to just "Headrush Beach," so the run produced a likely incorrect recommendation with little explanation.

Failure summary (judge, tier 2). The system relied on a flattened, unverified PDF table, failed to communicate or reconstruct the row-level availability/pool data, and confidently recommended Headrush Beach even though the aligned rental-house data indicates it was not available and had no pool.

Mechanisms flagged.

- *unfulfilled request*: The orchestrator asked FileSurfer for a summarized, comparative extraction, but FileSurfer returned raw flattened page text instead.
- *table flattening*: FileSurfer's extraction lost the table's row alignment by listing names, then ratings/vacancies, then pool values, then reviews separately.
- *lost structure*: The column headers and values were separated in a way that obscured which pool and vacancy values belonged to which accommodation.
- *missing summary*: FileSurfer did not explicitly identify which rental houses were available, despite that being central to the task.
- *missed verification*: The orchestrator proceeded directly from the ambiguous extraction to final selection instead of asking FileSurfer or another tool to reconstruct the table rows.
- *wrong availability*: The Assistant asserted that Headrush Beach was marked available, but the extracted rental-house row sequence indicates Headrush Beach corresponds to a "3 No" vacancy row when aligned in order.

Magentic-One 3cef3a44 (GAIA L1)

Task. I'm making a grocery list for my mom, but she's a professor of botany and she's a real stickler when it comes to categorizing things. I need to add different foods to different categories on the grocery list, but if I make a mistake, she won't buy anything inserted in the wrong category. Here's the list I have so far:

milk, eggs, flour, whole bean coffee, Oreos, sweet potatoes, fresh basil, plums, green beans, rice, corn, bell pepper, whole allspice, acorns, broccoli, celery, zucchini, lettuce, peanuts

I need to make headings for the fruits and vegetables. Could you please create a list of just the vegetables from my list? If you could do that, then I can figure out how to categorize the rest of the list into the appropriate categories. But remember that my mom is a real stickler, so make sure that no botanical fruits end up on the vegetable list, or she won't get them when she's at the store. Please alphabetize the list of vegetables, and place each item in a comma separated list.

Outcome. Original GPT-4o run: failed. Reproduction: failed (normalized exact match; expected answer: "broccoli, celery, fresh basil, lettuce, sweet potatoes"). Screened cat-2 prior: medium.

MAST modes (judge). 1.1, 1.3, 1.4, 1.5, 2.1, 2.2, 2.3, 2.6, 3.1, 3.2, 3.3

Run narrative (judge, tier 1). The run began with repeated model-mismatch warnings, then the orchestrator summarized the user's grocery-classification task and assembled a full multi-agent team despite planning to answer from knowledge alone. The Assistant initially produced a comma-separated list that included zucchini, a botanical fruit, and then repeated the same answer several times while the orchestrator asked for meta status instead of finalizing. The orchestrator then repeatedly regenerated long fact sheets and "root cause" plans that claimed the answer had not been delivered, gradually changed the classification policy, and identified several ambiguities for lookup, but never actually used WebSurfer or any verification tool. Across these revisions, zucchini was eventually removed, basil was excluded as an herb, but green beans remained in the final list despite the trace itself noting they are botanically seed pods and repeatedly stating that fruits/seeds/seed structures should be excluded. The final answer was formatted correctly but likely violated the user's strict botanical constraint by including green beans.

Failure summary (judge, tier 2). The system repeatedly re-planned and self-assessed instead of resolving the botanical classification, ultimately producing a final vegetable list that still included green beans despite recognizing them as seed pods under a strict botanical framing.

Mechanisms flagged.

- *lookup contradiction*: The initial fact sheet listed several botanical classifications as facts to look up, but the plan immediately said no lookup was likely needed.
- *loose classification*: The initial educated guesses were loose about botanical categories, treating green beans as likely vegetables while warning that other items required caution.
- *fruit included*: The Assistant's first answer included zucchini, which is a botanical fruit and directly conflicted with the user's warning.
- *repeated error*: The same incorrect list containing zucchini was requested and returned multiple times.
- *state misread*: The orchestrator later described the run as if no answer had been delivered, despite several prior Assistant answers.

- *meta looping*: The orchestrator repeatedly generated long self-diagnostic plans instead of simply correcting the classification and producing the final answer.

Magentic-One 3f57289b (GAIA L1)

Task. How many at bats did the Yankee with the most walks in the 1977 regular season have that same season?

Outcome. Original GPT-4o run: failed. Reproduction: failed (normalized exact match; expected answer: "519"). Screened cat-2 prior: low.

MAST modes (judge). 1.1, 2.5, 2.6, 3.1, 3.3

Run narrative (judge, tier 1). The system began with a straightforward plan to look up 1977 Yankees batting statistics, identify the team leader in walks, and return that player's at-bats. WebSurfer first searched Bing and exposed an apparently misleading snippet claiming Thurman Munson had 66 walks and 502 AB, but the orchestrator appropriately asked it to open Baseball-Reference for confirmation. After several redundant scrolling instructions, WebSurfer reached the Baseball-Reference Standard Batting table and returned the full visible table text, which showed Roy White with 75 BB and 519 AB, narrowly ahead of Reggie Jackson with 74 BB. However, WebSurfer did not synthesize the leader, and the orchestrator appears to have misread or ignored the table, finally answering only "595", which is Thurman Munson's AB total from the first row, not the AB total of the walks leader. The task therefore failed despite the correct data being present in the trace.

Failure summary (judge, tier 2). The system retrieved the Baseball-Reference batting table containing the correct walks leader, Roy White with 75 BB and 519 AB, but the orchestrator ignored/misread that data and prematurely returned Thurman Munson's 595 AB instead.

Mechanisms flagged.

- *missed verification*: The initial plan called for cross-checking if needed, but the final answer was given without any explicit cross-check after the correct table appeared.
- *misleading snippet*: The Bing results viewport contained an apparent answer-like snippet claiming Thurman Munson had 66 walks and 502 AB, but this was not verified and conflicts with the later Baseball-Reference table.
- *missing synthesis*: When WebSurfer finally displayed the Standard Batting table, it provided raw table text but did not identify the leader or answer the orchestrator's explicit question.
- *unused correct data*: The correct answer was present in WebSurfer's table output: Roy White had 519 AB and 75 BB.
- *missed comparison*: The table also showed that Reggie Jackson had 74 BB and 525 AB, making him close but not tied with Roy White; this tie/leader distinction was not reported.
- *wrong row selected*: The final answer appears to have selected Thurman Munson's AB total from the first table row, even though Munson did not lead the team in walks.

Magentic-One 5a0c1adf (GAIA L1)

Task. What is the first name of the only Malko Competition recipient from the 20th Century (after 1977) whose nationality on record is a country that no longer exists?

Outcome. Original GPT-4o run: failed. Reproduction: failed (normalized exact match; expected answer: “Claus”). Screened cat-2 prior: high.

MAST modes (judge). 1.1, 1.3, 2.2, 2.3, 2.5, 2.6, 3.1, 3.2, 3.3

Run narrative (judge, tier 1). The run began with a reasonable plan to identify post-1977 Malko Competition recipients and filter their recorded nationalities for defunct countries, but execution repeatedly failed at data collection. WebSurfer first hit Bing CAPTCHA pages, Assistant/ComputerTerminal briefly switched to DuckDuckGo and found useful leads including the official DR pages and Wikipedia, but those leads were not systematically exploited. WebSurfer then opened a modern DR article and the official malkocompetition.dk site, confirmed only the 2024 winners page and a South Korea entry, but never navigated the archive or crawled year pages despite repeated instructions. The orchestrator repeatedly summarized the same missing-data problem and proposed systematic extraction, but did not successfully delegate or execute it; later it pivoted to an apparently unrelated Wiener Philharmoniker URL, causing more navigation dead ends. In the end, after another ignored instruction to stop using Bing, the orchestrator produced the bare answer “Janos” without any source, candidate row, nationality evidence, or uniqueness verification in the trace.

Failure summary (judge, tier 2). The system failed to collect or verify the post-1977 Malko winners archive, repeatedly ignored direct navigation instructions, looped on planning/search dead ends, and prematurely returned an unsupported answer.

Mechanisms flagged.

- *scope ambiguity*: The initial plan treated “recipient” mostly as “winners/recipients,” creating a scope ambiguity that was never resolved; the official page later showed multiple prize recipients per year, not just first-prize winners.
- *blocked search*: WebSurfer’s first search used Bing and immediately hit a human-verification challenge, blocking progress.
- *incomplete fallback*: ComputerTerminal did not follow the fallback instruction after DuckDuckGo returned status 202 for the third query; it simply stopped without trying another search source.
- *dropped lead*: The Wikipedia result was explicitly found and then requested by the orchestrator, but WebSurfer did not open it and instead returned to Bing search.
- *instruction ignored*: WebSurfer disobeyed the instruction to open direct pages and avoid Bing by typing another Bing search query.
- *missed link*: When WebSurfer did open the DR article directly, it extracted only a modern 2024 winner and did not use the article’s own pointer to the official competition site.

Magentic-One 5d0080cb (GAIA L1)

Task. What was the volume in m^3 of the fish bag that was calculated in the University of Leicester paper ”Can Hiccup Supply Enough Fish to Maintain a Dragon’s Diet?”

Outcome. Original GPT-4o run: failed. Reproduction: solved (normalized exact match; expected answer: “0.1777”). Screened cat-2 prior: medium.

MAST modes (judge). 1.1, 2.4, 2.5, 2.6, 3.2, 3.3

Run narrative (judge, tier 1). The run began with model-mismatch warnings, then the orchestrator formed a reasonable plan to find the University of Leicester paper, inspect the PDF, extract the fish-bag calculation, and return the volume with a brief derivation. WebSurfer first found a Bing result whose generated snippet already contained the likely answer, 0.1777 m^3, but because it was not primary-source evidence the orchestrator continued to the University of Leicester article page. WebSurfer opened the article and the PDF viewer, but initially failed to inspect, search, scroll, or extract the bag-volume section, instead reporting only that no dimensions were visible on the current PDF page. After repeated prompts, WebSurfer downloaded the PDF, and the orchestrator handed the local path to FileSurfer. FileSurfer successfully extracted the relevant primary-source text, including the model assumptions, dimensions, calculation, and explicit statement that the bag capacity is 0.1777 m3. The orchestrator then gave the final answer as only "0.1777", which is substantively correct but omitted the unit, citation, and explanatory note that its own plan had called for.

Failure summary (judge, tier 2). The system found the correct primary-source volume but WebSurfer initially performed only superficial PDF inspection, and the orchestrator's final answer dropped the unit, quote, derivation context, and planned verification.

Mechanisms flagged.

- *plan not followed*: The orchestrator's initial plan required a fuller final response than was eventually given, including cross-checking and a brief note on how the number was obtained.
- *incomplete inspection*: When first sent to the PDF, WebSurfer did not actually inspect or search the PDF content; it only reported the sparse viewer page showing a title and download link.
- *missed content*: Despite being instructed to search for terms inside the PDF, WebSurfer only considered what was visible on the current page and concluded that no dimensions or volume were visible.
- *premature limitation*: WebSurfer narrowed the evidence to the introductory sack mention and abstract rather than continuing to the later calculation section requested by the orchestrator.
- *dropped context*: The paper's exact assumptions and dimensions were recovered by FileSurfer, but the orchestrator did not preserve them in the final answer.
- *unit omitted*: The final answer omitted the unit even though the user specifically asked for the volume in m^3.

Magentic-One 72e110e7 (GAIA L1)

Task. Under DDC 633 on Bielefeld University Library's BASE, as of 2020, from what country was the unknown language article with a flag unique from the others?

Outcome. Original GPT-4o run: failed. Reproduction: failed (normalized exact match; expected answer: "Guatemala"). Screened cat-2 prior: medium.

MAST modes (judge). 1.1, 1.2, 1.3, 2.1, 2.3, 2.4, 2.5, 2.6, 3.1, 3.2, 3.3

Run narrative (judge, tier 1). The system began with a reasonable plan to inspect BASE DDC 633, asked WebSurfer to search, clicked a DNB explanatory result, then followed its external link to BASE's Dewey browsing page. The BASE page rendered blank in the browser, after which

the orchestrator repeatedly instructed WebSurfer to wait or try to load the same page; WebSurfer only waited and never extracted source, refreshed, searched archives, or otherwise pivoted. The orchestrator then entered a long self-reflective loop, repeatedly rewriting fact sheets and plans that correctly diagnosed the need for archived/source-based inspection, but it did not actually delegate those new searches or use other tools such as ComputerTerminal. No DDC 633 record list, unknown-language article, flag metadata, title, or country was ever found, and the run ended with the final answer "unknown."

Failure summary (judge, tier 2). The system failed to answer the country question after the BASE page rendered blank, repeatedly waiting and rewriting plans instead of executing the stated archive/source-search strategy, then terminated with an unsupported unknown answer.

Mechanisms flagged.

- *blank page*: The BASE Dewey page opened but rendered blank, with no visible text or metadata.
- *repeated wait*: After the blank BASE page, the orchestrator repeatedly asked WebSurfer to load or inspect the same interface, but WebSurfer only waited and reported the same blank state.
- *instruction not followed*: WebSurfer did not follow the full instruction to interact with the blank page, refresh, scroll, accept cookies, or inspect source/links; it only waited.
- *unexecuted plan*: The orchestrator correctly recognized the failure mode and proposed archived/source-based strategies, but this insight was not translated into actual tool calls.
- *self-loop*: After diagnosing the issue, the orchestrator repeatedly generated revised fact sheets and plans instead of asking WebSurfer to perform the new searches.
- *missing delegation*: The orchestrator repeatedly stated that the next step was archive/source searching, but no subsequent WebSurfer search for those targeted archive queries appears in the trace.

Magentic-One 7673d772 (GAIA L1)

Task. On Cornell Law School website's legal information institute, under the fifth section of federal rules alphabetically, what word was deleted in the last amendment to the first rule in the article that has "witnesses" in the most titles as of 2021?

Outcome. Original GPT-4o run: failed. Reproduction: failed (normalized exact match; expected answer: "inference"). Screened cat-2 prior: medium.

MAST modes (judge). 1.1, 1.3, 2.2, 2.3, 2.5, 2.6, 3.1, 3.2, 3.3

Run narrative (judge, tier 1). The run began with the orchestrator planning to use Cornell LII's Federal Rules index, but it immediately treated the displayed fifth item as the "fifth section alphabetically" and proceeded into the Federal Rules of Bankruptcy Procedure without resolving the alphabetical ambiguity. WebSurfer first tried Bing and hit a human-verification challenge, then reached the Cornell page but mostly scrolled manually instead of systematically searching or extracting the TOC. It made an unsupported claim that Part VII/Rule 7031/"upon" was the answer, then wandered to unrelated rules 2007 and 2001; the orchestrator correctly flagged that as unverified but then itself recorded dubious "verified" facts about Rule 2001 and developed hunches

around Part II. Despite repeated instructions not to use search and to scan the TOC, WebSurfer again used Bing, then later returned to Cornell and produced a final unsupported claim that Part VII rules 7063–7071 contained "witnesses" in their titles even though the titles it quoted did not contain that word, asserted without opening or quoting the rule notes that Rule 7063's last amendment deleted "witnesses," and the orchestrator accepted that as the final answer.

Failure summary (judge, tier 2). WebSurfer ignored direct navigation and verification instructions, relied on unsupported and internally contradictory witness-title claims, and the orchestrator prematurely accepted an unverified final answer.

Mechanisms flagged.

- *ambiguous routing*: The initial fact sheet treated the phrase "fifth section of federal rules alphabetically" as unresolved, but later the system effectively chose Bankruptcy from page order rather than proving alphabetical order.
- *ignored instruction*: The orchestrator correctly redirected WebSurfer away from Bing, but this instruction had to be repeated multiple times later.
- *alphabetical error*: The visible Federal Rules index listed Bankruptcy fifth in display order, but not alphabetically; the run treated it as the answer to the alphabetical clue without resolving that mismatch.
- *premature selection*: WebSurfer clicked Bankruptcy immediately when asked for the fifth alphabetical section, without explaining or verifying the alphabetical ordering.
- *date mismatch*: The Bankruptcy page was current through 2025, while the user asked about the state "as of 2021"; no version control or historical 2021 text was used.
- *incomplete scan*: Instead of using browser find, page text extraction, or a script, WebSurfer manually scrolled a small portion of a long TOC, which was inadequate for exhaustive counting.

References

[1] M. Cemri et al., "Why Do Multi-Agent LLM Systems Fail?" arXiv:2503.13657, 2025.